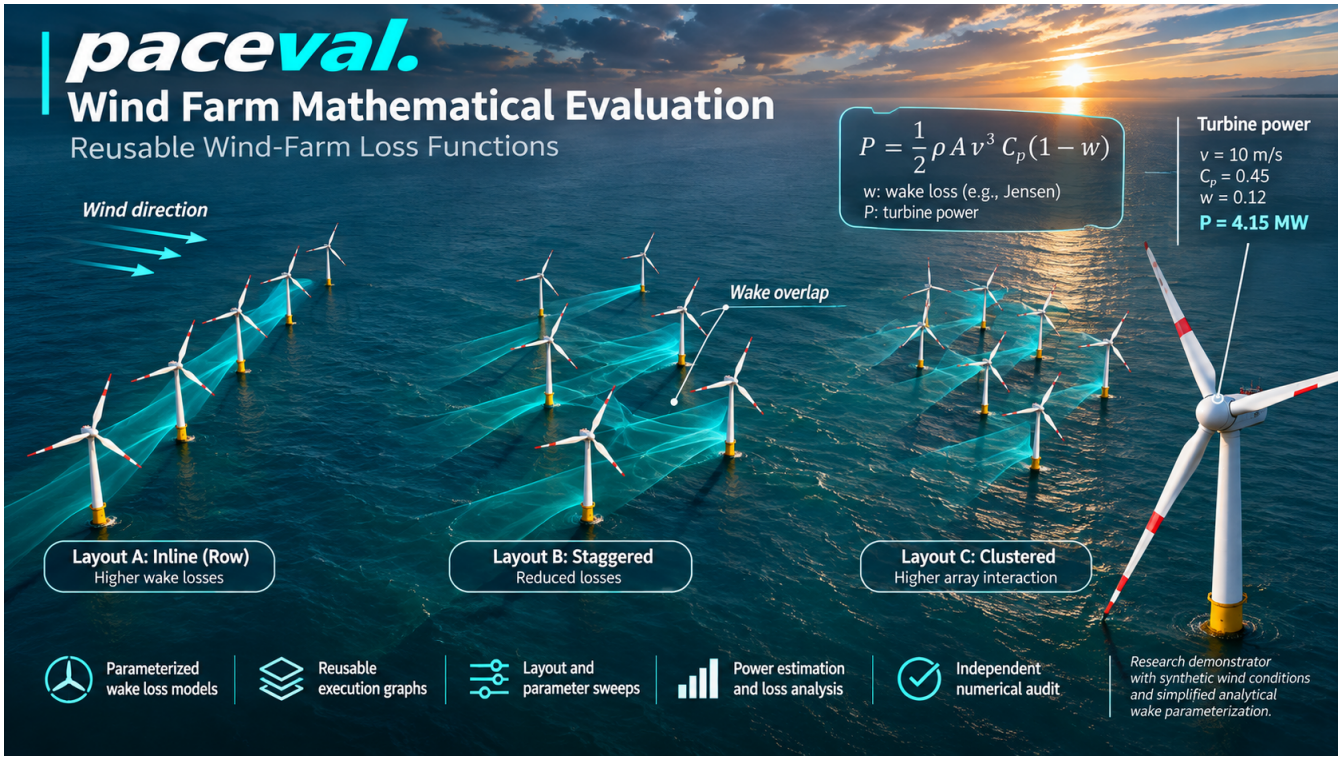
Reusable Wind-Farm Loss Functions

A reproducible numerical demonstrator with paceval execution graphs



Research contribution

The WP-015 research proposal is implemented as a working C++ study of three synthetic eight-turbine wind-farm layouts. One hundred parameterized wake expressions are compiled into reusable paceval computation graphs and evaluated across 32 wind conditions per layout. Aerodynamic power is computed by an additional reused graph; numerical outputs are checked against independent C++ and high-precision references.

Study size	Verification
100 wake models; 3 layouts; 32 wind cases	240,000 native wake checks; 153,600 power checks
384 high-precision expression samples	All within specified tolerance
24 raw TINC interval samples	24 reference inclusions on a separate smooth control kernel

The experiment demonstrates repeatable mathematical evaluation, not independently validated wind-farm physics, site yield predictions or a claimed performance advantage.

1. Research question and execution boundary

Can many explicit wind-wake loss expressions be compiled, versioned and evaluated repeatedly while preserving known mathematical semantics and reproducible wind-farm scenarios? This demonstration answers the execution-feasibility question for a precisely defined synthetic model.

What paceval executes

Exactly 100 parameterized nonlinear wake-deficit functions containing square roots, rational terms and exponentials are instantiated through paceval_CreateComputation. A separate graph evaluates cubic turbine power. Batch evaluation uses paceval_dGetComputationResultExt; graph deletion and runtime lifecycle use the original SDK facade.

What the C++ host executes

The host rotates turbine coordinates into the wind direction, finds upwind/downwind pairs, applies square-sum wake superposition, computes effective rotor inflow, caps rated power, aggregates farm power and manages model versions and outputs. No hidden optimization or calibrated physical solver is claimed.

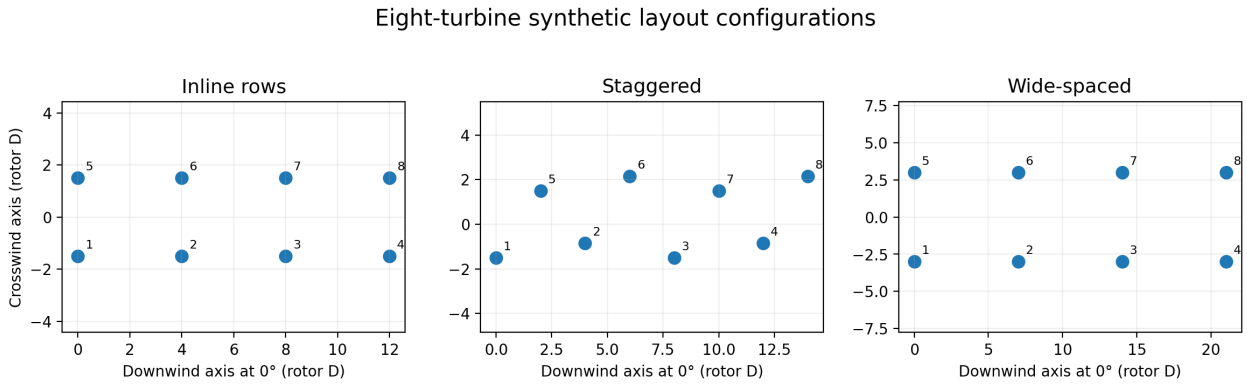


Figure 1. Three eight-turbine layouts in rotor-diameter units, generated directly from the source coordinates.

The layouts are deliberately designed for a transparent mathematical comparison. Their relative losses should not be extrapolated to a real wind project.

2. Explicit wind-energy model

Rotor power and units

P_raw [MW] = 0.5 * rho [kg/m3] * A [m2] * Cp * v_eff^3 / 1,000,000

The rotor diameter is 140 m, swept area A = pi * (70 m)^2, reference density 1.225 kg/m3, model Cp approximately 0.40 to 0.428, and the illustrative power cap is 5 MW per turbine. A bounded synthetic inflow range of 6–12 m/s is tested.

Pairwise deficit function (simplified wake surrogate)

delta(dx,dy) = (1 - sqrt(1-Ct)) * [D/(D+2*k*dx)]^2 * exp[-0.5*(dy/(D/2+k*dx))^2]

Here dx is strictly positive streamwise separation, dy is signed crosswind separation, D is rotor diameter, Ct is a coefficient of turbine thrust and k is wake expansion. The equation combines a Jensen-inspired radial expansion factor with a smooth Gaussian crosswind taper; it is not the exact Jensen top-hat, PyWake NOJ or FLORIS Gaussian model.

Superposition and delivered turbine power

v_eff = v_inf * max(0.20, 1 - sqrt(sum_j delta_j^2)); P = min(5 MW, P_raw)

The square-sum overlap rule, 20% lower inflow floor, static Ct and power cap are explicit host assumptions. The raw power loss caused by velocity deficit is accounted for by computing power from v_eff rather than by separately multiplying by another loss coefficient.

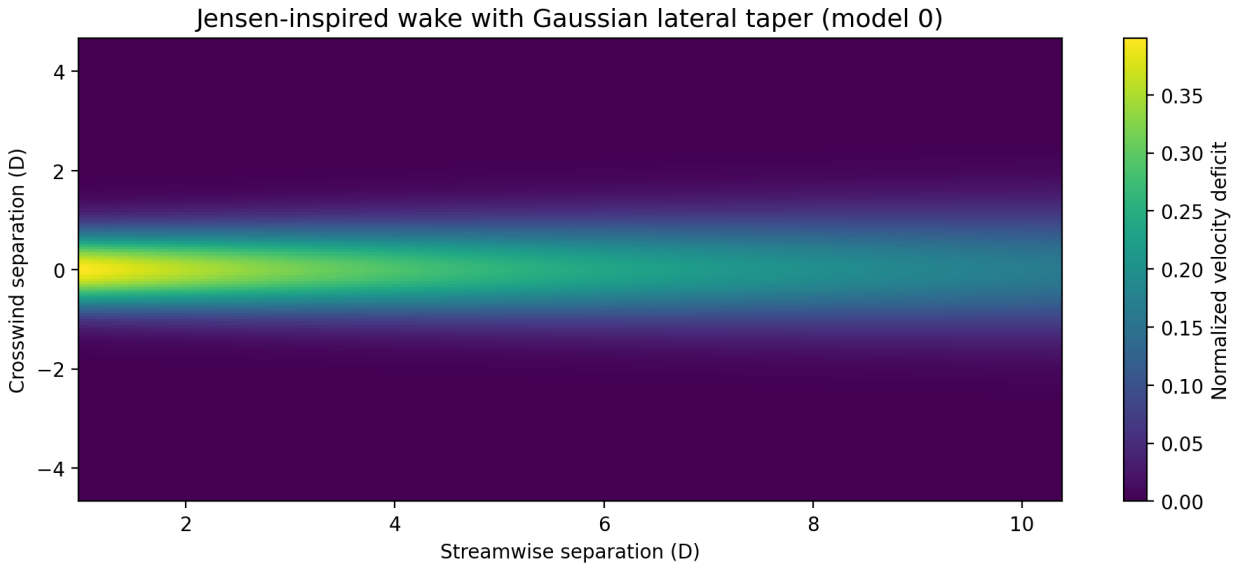


Figure 2. Analytical wake deficit for one selected coefficient set; not a CFD flow field or measured wake.

3. Defined workload and versioned graphs

Component	Controlled configuration
Turbines and layouts	Eight turbines per layout; inline, staggered and wide-spaced
Wind conditions	4 free-stream speeds x 8 directions = 32 conditions
Graph variations	100 coefficient versions with distinct Ct and k; Cp also varies
Wake and power execution	Pairwise deficits, effective inflow, gross and delivered MW
Graph update	Model 17: Ct is changed; model 18 retained and re-evaluated unchanged

Wind conditions, rotor geometry and all formula parameters are deterministic. The experiment contains no imported SCADA or wind-resource measurements, no annual-energy extrapolation and no site-specific frequency weighting.

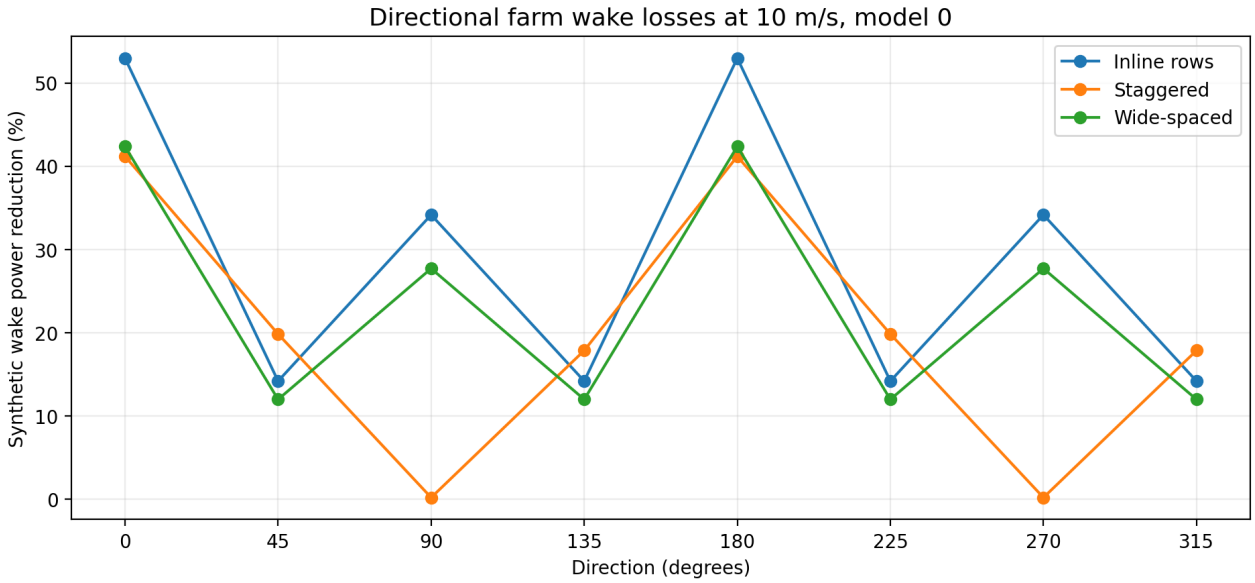


Figure 3. Directional losses for a selected model at 10 m/s, calculated from published farm_results.csv.

4. Farm power and model sensitivity

Under the deliberately simplified model, the effect of layout and prevailing direction is evident without invoking a neural network or an external CFD package. For model 0 at 10 m/s and a 0-degree wind direction, net illustrative power is 14.190 MW for inline rows, 17.747 MW for staggered turbines and 17.388 MW for the wide-spaced layout.

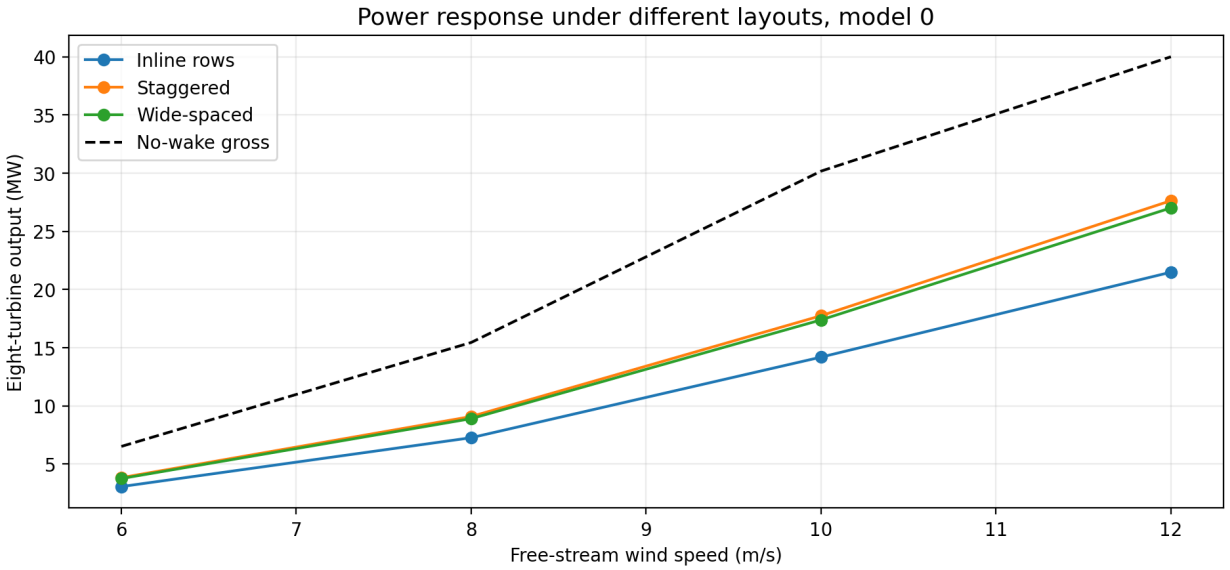


Figure 4. Net farm power versus free-stream speed at 0 degrees; dashed reference is hypothetical no-wake gross power.

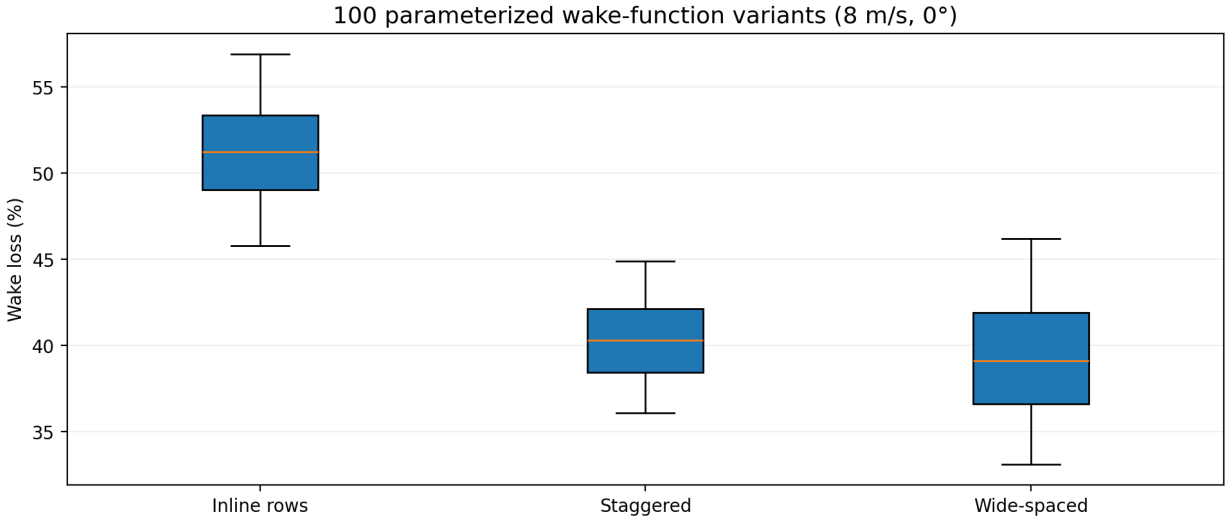


Figure 5. Spread of synthetic wake losses across the 100 separately compiled parameter variations.

5. Per-turbine effects and controlled changes

The 100 graphs are not one static formula re-evaluated 100 times. Each uses a unique coefficient set and is compiled into a distinct paceval computation handle. The mathematical kernels are then reused across all layouts, wind directions and wind speeds. Model 17 is deliberately updated to version 2 while model 18 continues unchanged.

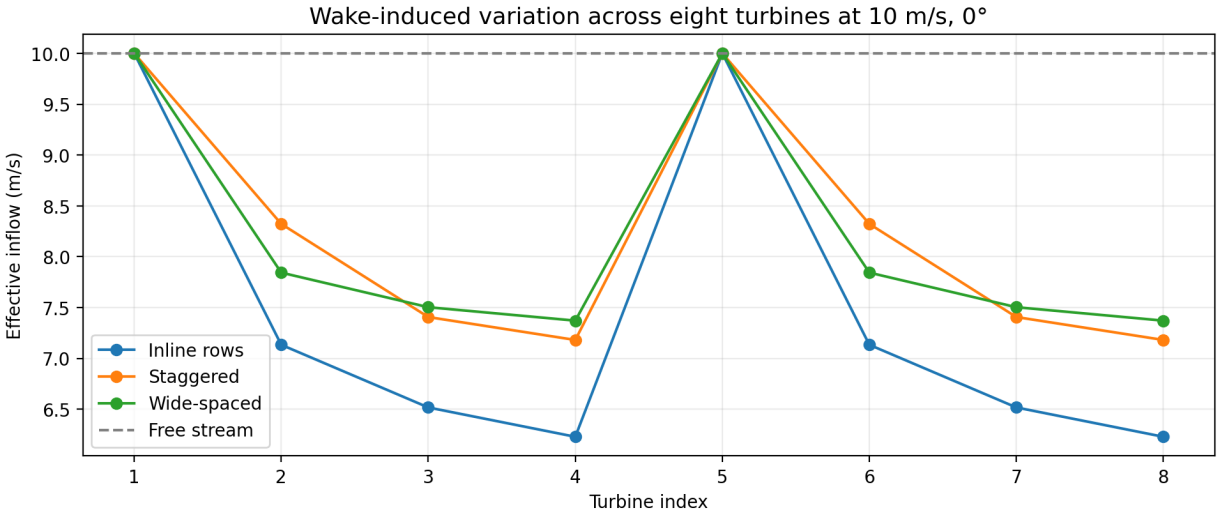


Figure 6. Effective inflow for each of eight turbines across the three layouts at 10 m/s and 0 degrees.

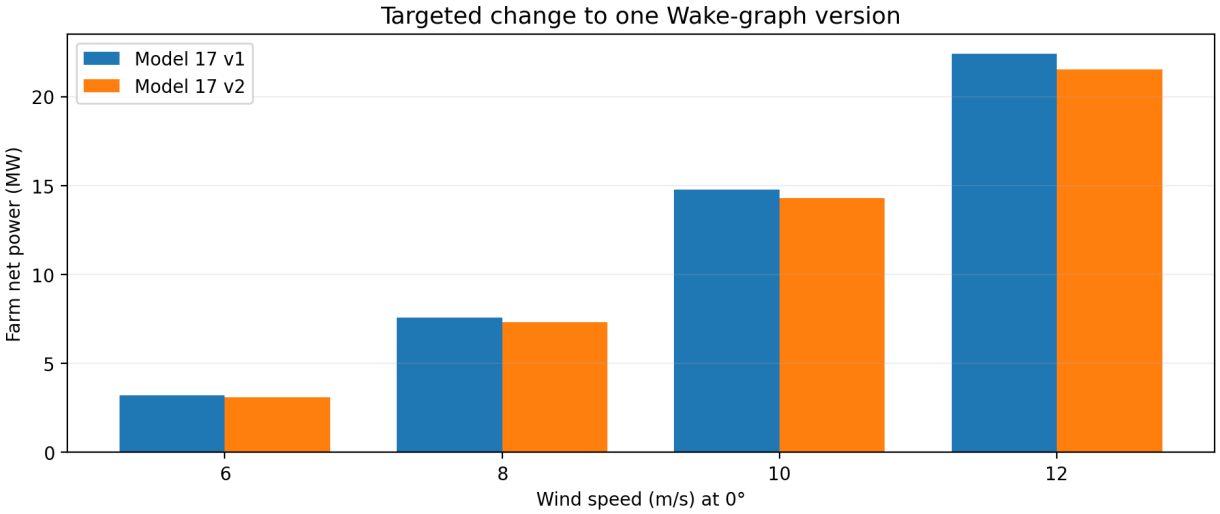


Figure 7. Replacing one versioned graph changes only the targeted model; the other graph handles remain live.

6. Numerical validation and independent audit

Validation layer	Measured study result
Native C++ reference for wake pairs	240,000 / 240,000 within tolerance
Native C++ reference for cubic power	153,600 / 153,600 within tolerance
Independent 110-digit samples	384 / 384 within tolerance
Max absolute error in 384 independent samples	3.58 x 10^-16 (dimensionless or MW depending on kernel)
Aggregation consistency	9,600 / 9,600 farm summaries validated
Raw paceval TINC control intervals	24 / 24 contain the exact binary-input reference

The audit script reconstructs input doubles from their 17-digit CSV representation and computes the mathematical reference independently with mpmath (110 decimal digits). A separate smooth polynomial TINC graph cp*v^3, with dyadic binary-exact inputs, verifies 24 raw intervals. These observations do not establish universal outward-rounding or a formal bound over arbitrary input domains.

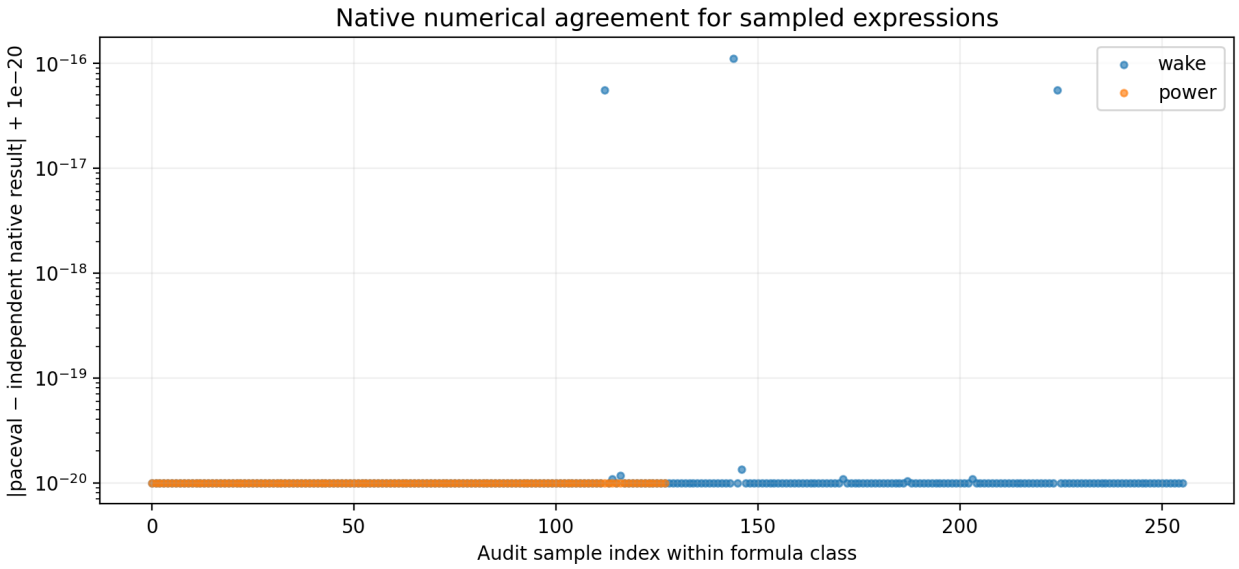


Figure 8. Absolute native comparison differences in selected outputs; the plotted small offset visualizes exact-zero residuals on the logarithmic axis.

7. Reproducibility and scientific interpretation

Build and execution

```
cmake -S src -B build -DPACEVAL_RUNTIME=/absolute/path/to/paceval.so
cmake --build build -j
ctest --test-dir build --output-on-failure
./build/wind_farm --variants 100 --batch 128 --out results
python3 src/high_precision_reference.py results
```

The project contains src/wind_farm.cpp, src/CMakeLists.txt, the official SDK facade src/paceval_main.h/cpp, replayable input and output CSV files, high_precision_reference.py, make_figures.py and make_whitepaper.py. A compatible external Linux library is required; it is not included. The exact tested setup was Linux x86-64 with GCC 14.2 and Clang 17, alongside AddressSanitizer/UndefinedBehaviorSanitizer runs. Each compiler build passed six CTest checks.

Interpretation

A successful mathematical graph-execution test does not validate the underlying wake model, a wind farm annual energy yield, turbulence, rotor averaging, yaw deflection, optimization or grid dispatch. The present evidence is sufficient for a controlled proof of repeatable mathematical loss-function evaluation. An independent PyWake/FLORIS model and site observations would be necessary for an application-level physics comparison.

Selected sources

1. paceval C/C++ mathematical engine and examples: https://github.com/paceval/paceval

2. DTU PyWake engineering wind farm models, including NOJ/Jensen: https://topfarm.pages.windenergy.dtu.dk/pywake/api/PredefinedEngineeringWindFarmModels.html

3. DTU PyWake wake superposition methods: https://topfarm.pages.windenergy.dtu.dk/pywake/notebooks/SuperpositionModels.html

4. NREL FLORIS wind farm modeling tutorial: https://www.nrel.gov/docs/libraries/wind-docs/systems-engineering-workshop-2019-floris.pdf

5. Source proposal: WP_015_Reusable_Wind_Farm_Loss_Functions.pdf (not republished in this GitHub package).

Publication statement

No execution-time comparisons, energy savings, customer benefits or turbine farm forecasts are asserted. This report describes a tested synthetic research demonstrator using a controlled explicit mathematical model. Original SDK license notices are preserved; use of the external runtime remains subject to its own terms.